

1. Describe al menos dos medidas prácticas para implementar un enfoque de código seguro en una API REST

Respuesta:

- Una medida es implementar en la autenticación con tokens seguros como JSON Web Tokens, para autenticar a los usuarios. Así como definir roles y permisos a los usuarios para evitar acceder a información sensible a usuarios no autorizados.

También si se requiere se puede añadir el protocolo OAuth2.0 para delegar la autenticación y autorización entre múltiples servicios o integración con terceros.

- Otra medida es usar HTTPS para que los datos estén cifrados durante el tránsito, usando certificados SSL/TLS
- Por último y más importante limpiar y dar formato a los datos ingresados por el usuario para evitar ataques de inyección SQL o cualquier otro código malicioso.

2. Menciona tres prácticas esenciales para garantizar la calidad del código en una revisión (code review)

Respuesta:

- Una buena práctica es mantener un código claro, estructurado y fácil de entender, por ejemplo, en C# podemos usar PascalCase para métodos y clases, y camelCase para variables, evitar métodos demasiado largos y complejos, comentar el código de forma concisa cuando sea necesario.

- Otra práctica es la revisión del rendimiento del nuevo código, evitando introducir cuellos de botella. Por ejemplo, evitando consultas a la base de datos innecesarias y/o mal optimizada. Optimizar la parte del código que consume demasiados recursos evitando bucles innecesarios o consultas múltiples en las listas.

- Muy importante también es el manejo de errores, manejando adecuadamente las excepciones, sin mostrar información sensible en el error, haciendo un registro de ellos, así como un manejo de errores de manera global para errores no controlados. Por eso es importante hacer una correcta validación de las entradas para que el usuario o alguien mal intencionado ingrese datos que causen errores en el sistema.

3. Explica cómo implementarías un manejo de errores centralizado en una API REST desarrollada con .NET Core. Incluye conceptos como middleware o logging

Respuesta:

Usando un Middleware podemos capturar excepciones no controladas y devolver respuestas en formato JSON.

Paso 1, configuramos el archivo appsettings.json para configurar el logging:

* Ver clase en el archivo github

Paso 2, Implementar el Middleware de manejo de errores, creando una clase `ExceptionHandlerMiddleware` en la carpeta `Middleware`:

* Ver clase en el archivo github

Paso 3, Registrar el middleware en el archivo `program.cs`

```
app.UseMiddleware<ExceptionHandlerMiddleware>();
```

Paso 4, Habilitar el logging en `program.cs`

```
var builder = WebApplication.CreateBuilder(args);  
builder.Logging.ClearProviders();  
builder.Logging.AddConsole(); // Log en la consola
```

Paso 5, Crear un controlador para probar errores de nombre `TestController.cs`

* Ver clase en el archivo github

Paso 6, Ejecutar la aplicación, y usar el navegador para probar en la siguiente dirección cambiando el puerto al requerido,

<https://localhost:5039/api/test/error>

Tendrá como respuesta un json con el error, y el log en la consola.

Esta pregunta esta desarrollado listo para ejecutar en github compartido listo con este documento.

4. ¿Cómo guiarías a un desarrollador junior en la solución de un problema técnico sin resolverlo directamente?

Respuesta:

Guiar al desarrollador junior a que encuentre la solución por sí mismo fomentando su capacidad de resolver problemas de la siguiente manera:

Primero, debe entender el problema, le pediría que describa el problema con sus propias palabras, luego que lo desglose en partes más pequeñas para que pueda encontrar el punto o patrón que causa el problema.

Segundo, fomentar a que investigue, en la programación hay mucha información en internet, haciendo la pregunta correcta se encuentra fácil la solución, por eso es importante el paso anterior.

Tercero, luego de encontrar alternativas, debe probar esos enfoques y encontrar las ventajas y desventajas de cada uno.

Cuarto, una vez tenga la solución debe de implementar, pero manteniendo la claridad y eficiencia del código respetando los estándares de la organización.

Quinto, por último, debe de aprender y documentar para que pueda resolverlo fácilmente el u otro programador, además de motivarlo para que actúe igual para futuros casos.

5. Un sistema de comercio electrónico utiliza Azure Functions para procesar pedidos en tiempo real. Responde:

a. ¿Cómo garantizarías que cada pedido sea procesado exactamente una vez?

Respuesta:

Una de las formas es usando un identificador único del pedido, por ejemplo, idOrder, antes de procesar el pedido se verifica si existe en la base de datos, si es así se ignora y se marca como duplicado.

También se puede usar Azure service Bus para el manejo de colas, y se puede aplicar el método EnableDuplicateDetection , para evitar que mensajes duplicados lleguen a la cola.

b. Menciona dos estrategias para manejar errores transitorios

Respuesta:

Primera estrategia sería implementar un patrón de reintentos, consiste

en reintentar automáticamente una operación fallida después de un intervalo de tiempo.

Se puede hacer configurando un el Service Bus Trigger, se coloca el máximo número de intentos y el intervalo. O mediante código usando una librería como Polly, colocando un intervalo de tiempo exponencial para no saturar el sistema.

Otra estrategia es usar cola de mensajes fallidos, para aislar mensajes problemáticos y reprocesarlos posteriormente, Azure service bus permite colas de mensajes fallidos, si un mensaje no se procesa service bus lo mueve a una cola de mensajes fallidos. Se procede a configurar el número máximo de reintentos y el tiempo de vida del mensaje (TTL) en la cola, y luego crear una función separada para procesar los mensajes fallidos.

c. ¿Qué configuración implementarías para manejar picos de 100,000 transacciones por minuto?

Respuesta:

Debemos usar colas de mensajes para desacoplar la recepción de pedidos del procesamiento real, se realiza usando Azure service bus, habilitando el particionamiento para distribuir la carga, aumentando el número de unidades de mensajería, y que el tamaño de cola sea suficiente para manejar los picos.

SECCION 2: DESARROLLO PRACTICO

Desarrolla las siguientes funcionalidades para un sistema de gestión de órdenes de compra. Utiliza .NET Core para implementar los endpoints. (API REST)

1.- Desarrollo de EndPoints

- * Solución del proyecto en el archivo github.

2.- Garantías de calidad

- * Revisar en el archivo del proyecto github

3.- Análisis de la solución:

Este al ser un proyecto de prueba, la persistencia de datos se almacena en memoria, para un proyecto real se debe usar una base de datos y agregar más validaciones de acuerdo con los requisitos del negocio, así como un adecuado manejo de transacciones en código y en base de datos.

Para verificar los resultados se debe crear un producto haciendo un post con este formato:

POST http://localhost:5094/api/products

Content-Type: application/json

```
{  
  "name": "Laptop",  
  "price": 1200.00,  
  "stock": 10  
}
```

Esto devolverá una respuesta con los detalles del producto creado

Crear una orden:

POST http://localhost:5094/api/orders

Content-Type: application/json

```
{  
  "1": 2  
}
```

Esto devuelve una respuesta con los detalles de la orden creada

Y para consultar órdenes

GET http://localhost:5094/api/orders

Se obtendrá una lista de órdenes con sus detalles.

SECCION 3: COMUNICACION TECNICA

Tu equipo está planeando migrar una aplicación monolítica en ASP.NET MVC hacia microservicios

1.- Describe los pasos principales que seguirías para realizar esta migración

Respuesta:

Pasos principales serian:

Primero:

Análisis de la aplicación monolítica, identificando sus módulos, dependencias y

flujos de trabajo. Decidir qué funcionalidades se convertirán en microservicios.
Priorizar los servicios más críticos o independientes

Segundo:

Definir el entorno de desarrollo con las herramientas necesarias, incluyendo Docker, un sistema de CI/CD como Azure Devops. La plataforma para los microservicios, y el protocolo de comunicación ejem REST.

Tercero:

Desarrollar los microservicios, moviendo el código relacionado al microservicio asegurándose de que no haya dependencias no resueltas

Cuarto:

Pruebas unitarias y de integración, asegurándose el correcto funcionamiento, pruebas de carga para verificar que soporte el tráfico esperado y un monitoreo del rendimiento.

Quinto:

Documentar la nueva arquitectura y los microservicios, y repetir los pasos anteriores para los demás módulos de la aplicación, y optimizar según los resultados obtenidos.

Sexto:

A medida que se va migrando a microservicios, se procede a desactivar funcionalidades obsoletas de la aplicación monolítica. Validar la migración este completa y ajustar la infraestructura para reducir costos.

2. Explica cómo comunicarías al equipo las ventajas y los riesgos de esta transición.

Respuesta:

Como primer punto procedo a explicar por qué se está considerando esta transición.

Por ejemplo:

"Nuestra aplicación actual ha crecido en complejidad, lo que dificulta su mantenimiento y escalabilidad."

"Necesitamos una arquitectura más flexible para responder a las demandas del negocio y mejorar la experiencia del usuario."

Luego propongo el objetivo:

"Queremos mejorar la escalabilidad, reducir los tiempos de despliegue y facilitar la incorporación de nuevas funcionalidades."

Segundo punto menciono las ventajas de la transición:

"Cada microservicio puede escalarse de manera independiente según la demanda, lo que optimiza el uso de recursos."

"Los equipos pueden trabajar en paralelo en diferentes servicios, acelerando el tiempo de entrega."

"Al dividir la aplicación en servicios más pequeños, es más fácil entender, probar y mantener el código."

"Un fallo en un servicio no afectará necesariamente a toda la aplicación."

"Podemos desplegar actualizaciones de manera más rápida y segura, sin afectar toda la aplicación."

Tercero les menciono algunos riesgos:

"Gestionar múltiples servicios implica un mayor esfuerzo en infraestructura, monitoreo y gestión de errores."

"La comunicación entre servicios puede introducir latencia y requerir un diseño cuidadoso para evitar cuellos de botella."

Y seguidamente como mitigar los riesgos:

"No migraremos todo de una vez. Comenzaremos con un servicio piloto y aprenderemos de esa experiencia."

"Fomentaremos una cultura de colaboración entre desarrollo y operaciones para manejar la complejidad operativa."

Adicionalmente puedo mencionar casos de éxito:

"Compañías como Netflix, Amazon y Spotify han migrado exitosamente a microservicios, logrando mayor escalabilidad y velocidad de desarrollo."